

CPU Scheduling Simulator

Round Robin vs SJF vs SRTF

Table of Contents

1. [Project Overview](#)
2. [Requirements & Specifications](#)
3. [System Architecture](#)
4. [Scheduling Algorithms Implemented](#)
 - Round Robin (Preemptive, Time-Slicing)
 - SJF – Non-preemptive Shortest Job First
 - SRTF – Preemptive Shortest Remaining Time First
5. [Graphical User Interface](#)
6. [Test Scenarios & Results](#)
7. [Analysis & Comparison](#)
8. [Conclusion](#)

1. Project Overview

The **CPU Scheduling Simulator** is a Java Swing application that visually compares three fundamental scheduling algorithms:

- **Round Robin (RR)** – preemptive, time-sliced sharing.
- **Shortest Job First (SJF)** – non-preemptive, selects the job with the smallest burst time.
- **Shortest Remaining Time First (SRTF)** – preemptive version of SJF.

The purpose is to study differences in *average waiting time*, *turnaround time*, *response time*, fairness, and behaviour under various workloads. Users can input processes dynamically, set a time quantum, run the simulation, and instantly see Gantt charts, metrics tables, and a detailed comparison.

2. Requirements & Specifications

Functional Requirements (All Met)

- Dynamic process input via table – add/remove/edit rows at runtime.
- Time quantum field (for Round Robin).
- Input validation: duplicate PIDs, negative arrival, zero/negative burst, invalid quantum.
- Three scheduling algorithms: Round Robin, SJF (non-preemptive), SRTF (preemptive).
- Separate Gantt charts for each algorithm.
- Detailed results tables: PID, arrival, burst, completion, TAT, WT, RT, and averages.
- Round Robin ready-queue history view.
- Comparison summary and final conclusion panels.
- Pre-loaded test scenarios covering normal, short-job heavy, fairness, long-job sensitivity, and invalid input.

Non-functional Requirements

- Clean, responsive Swing GUI with tabs and scroll-safe layout.
- All labels and messages in English.
- Stable scheduling logic – no deadlocks or infinite loops.

3. System Architecture

The application follows a simple Model-View-Controller (MVC) pattern:

- **Model:** `Process.java` , `SchedulerResult.java` , `GanttEntry.java` – data structures.
- **View:** `MainFrame.java` , `GanttPanel.java` , `ResultsTablePanel.java` , `ReadyQueuePanel.java` – GUI components.
- **Controller:** `RoundRobinScheduler.java` , `SJFScheduler.java` , `SRTFScheduler.java` – algorithm logic.

The main entry point is `Main.java` , which launches `MainFrame` .

Key Classes

Class	Responsibility
Process	Stores process attributes and provides a <code>copy()</code> method for safe simulation.
GanttEntry	Represents a time slice in a Gantt chart (PID, start, end).
SchedulerResult	Aggregates process list, Gantt chart, average metrics, and RR queue history.
GanttPanel	Custom Swing panel that draws color-coded bars for a scheduling timeline.
ResultsTablePanel	Displays per-process metrics in a JTable and average summary.
ReadyQueuePanel	Text area showing the Round Robin ready queue at each time step.
RoundRobinScheduler	Implements RR with queue rotation, records queue states.
SJFScheduler	Non-preemptive SJF: selects smallest burst among arrived processes.
SRTFScheduler	Preemptive SJF: runs in 1-unit steps, always picks the process with the smallest remaining time.

4. Scheduling Algorithms – Detailed Logic

4.1 Round Robin (Preemptive)

Processes are added to a FIFO ready queue as they arrive. The CPU runs the process at the head for **up to quantum** time units. If the process finishes, it is removed; otherwise it returns to the tail. New arrivals during execution are added to the queue. When the queue is empty, the CPU idles until the next arrival.

Formulas used:

- $\text{Response Time} = \text{first start time} - \text{arrivalTime}$
- $\text{Turnaround Time} = \text{completionTime} - \text{arrivalTime}$
- $\text{Waiting Time} = \text{turnaroundTime} - \text{burstTime}$

Ready Queue Recording:

At each time step (before selecting a new process) the contents of the ready queue are captured and displayed in the dedicated panel.

4.2 SJF – Non-preemptive

At every scheduling decision (initially or after a process completion), the scheduler looks at all processes that have arrived and have remaining time > 0 . It picks the one with the smallest **original burst time** (arrival order tie-breaker). The selected process runs to completion without interruption.

4.3 SRTF – Preemptive

The simulator progresses in discrete time units. After each unit:

1. Newly arrived processes are added to a ready list.
2. The ready list is scanned every tick – the process with the smallest remaining time is chosen.
3. If a newly arrived process has a shorter remaining time than the currently running one, a context switch occurs.
4. Execution continues until all processes are completed.

The resulting Gantt chart is merged to combine consecutive identical-PID segments for clarity.

5. Graphical User Interface

The main window is divided into several logical sections:

1. **Top Control Panel:** Quantum field, action buttons (Add, Remove, Run, Clear), and five scenario-loading buttons.
2. **Process Input Table:** Editable JTable for PID, Arrival Time, Burst Time.
3. **Algorithm Tabs:** Three tabs (Round Robin, SJF, SRTF), each containing a Gantt chart (top) and a metrics results table (bottom).
4. **Ready Queue Panel:** Displays the Round Robin ready queue history after simulation.
5. **Bottom Tabs:** "Comparison Summary" and "Final Conclusion" text areas.

All panels are placed inside a scrollable vertical box to guarantee nothing is hidden on smaller screens.

6. Test Scenarios & Expected Results

Five pre-built scenarios are available via buttons. Below are the inputs and expected outputs.

Scenario A – Normal Mixed Workload

PID	Arrival	Burst
1	0	5
2	1	3
3	2	8
4	3	6

Quantum = 2

Expected **Round Robin** Gantt (simplified):

P1(0-2), P2(2-4), P3(4-6), P4(6-8), P1(8-10), P3(10-12), P4(12-14), P1(14-15),
P3(15-17), P4(17-19), P3(19-21)

Metrics for RR: avg WT $\approx$ 7.75, avg TAT $\approx$ 13.25, avg RT = 1.5

Non-preemptive SJF: avg WT ≈ 5.25 , avg TAT ≈ 10.75 , avg RT = 5.25

SRTF (Preemptive): avg WT = 5.0, avg TAT = 10.5, avg RT = 0.5

Observation: SRTF yields the lowest waiting time and best response time, at the cost of more context switches. RR gives the most balanced individual waiting times.

Scenario B – Short-Job Heavy

PID	Arrival	Burst
1	0	2
2	0	1
3	0	3
4	0	2
5	0	1
6	0	5

Quantum = 2

RR avg WT: 5.0 | **SJF avg WT:** 3.67 | **SRTF avg WT:** 3.67 (identical to SJF because all jobs arrive together).

Scenario C – Fairness Case

PID	Arrival	Burst
1	0	5
2	0	5
3	0	5
4	0	5

RR: WT spread = 12-15 (balanced), avg 13.5

SJF/SRTF: WT for first process = 0, last = 15 (unbalanced)

Scenario D – Long-Job Sensitivity

PID	Arrival	Burst
1	0	20
2	1	2
3	2	2
4	3	2
5	4	2

Quantum = 3

Round Robin: Long job (P1) waiting time = 5, short jobs finish quickly.

SJF: Long job runs uninterrupted → P1 WT = 0, but short jobs suffer huge WT (19+).

SRTF: Short jobs preempt the long job → P1 WT = 8, short jobs get low WT.

Scenario E – Invalid Input

Duplicate PID (1,1), negative arrival, zero burst, quantum = 0. Clicking Run shows successive error dialogs.

7. Analysis & Comparison

The application automatically generates a comparison summary after each run. It answers the following project-required questions:

Analysis Question	Typical Observation
Which algorithm gave lower average waiting time?	Usually $SRTF < SJF < RR$ (SRTF is optimal for waiting time).
Which gave lower average response time?	RR or SRTF often give very low RT because they start processes quickly; SJF (non-preemptive) may delay long processes but still gives instant start for the first one.
Did Round Robin appear fairer?	Yes – RR distributes CPU time evenly, preventing starvation. In the fairness scenario, all RR waiting times were within a narrow range.
Did SJF complete short jobs more efficiently?	Absolutely. Both SJF and SRTF minimise average waiting time by finishing short jobs early.
How did the chosen quantum affect RR?	Small quantum → more context switches, better responsiveness but higher overhead. Large quantum → behaves closer to FCFS. The simulator shows this directly in the Gantt chart and ready queue.
Which algorithm would you recommend?	Depends on the workload: interactive systems → RR; batch processing with known short jobs → SJF/SRTF. The conclusion panel suggests based on the current numbers.

8. Conclusion

By simulating all three algorithms side-by-side, users can clearly see the trade-offs:

- **Round Robin** ensures fairness and good response time – ideal for time-sharing systems.
- **SJF (non-preemptive)** minimises average waiting time but may cause starvation of long jobs.

- **SRTF (preemptive)** improves waiting time over SJF and offers the best responsiveness, at the cost of increased context switches.

The application's test scenarios and live recalculation make it an effective educational tool for operating system scheduling concepts.

Documentation generated for the CPU Scheduling Simulator project.